

# Catan Game Project Gen3

## Technical Overview v5 - From Initial Placement to Strategy-Guided Gameplay

Anthony van Tilburg

Prepared for GitHub publication

June 2026

### Version 5 milestone

Version 5 expands the project from an Initial Placement-focused Catan simulator into a strategy-guided gameplay engine. Human players can interact through the Execution phase, while AI players use a BEST NOW action recommendation loop backed by legal-action scanning, strategy matching, and same-turn continuation.

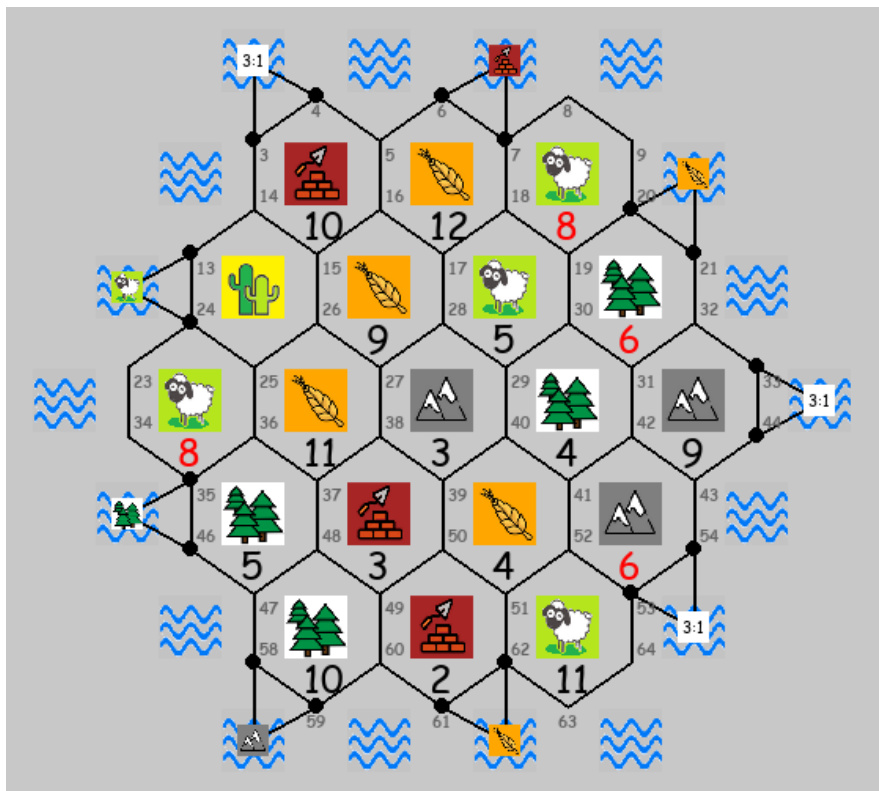


Figure 1. Existing Gen3 playboard rendering.

# Contents

1. Introduction
  2. Project status in v5
  3. System architecture
  4. GUI and interaction design
  5. Human gameplay flow
  6. Layered execution model
  7. Strategy and timing engine
  8. Trade with Bank and ports
  9. Development-card dashboard
  10. AI execution flow
  11. Sound and animation polish
  12. Known limitations and next steps
- Annex A. Module overview
- Annex B. Key classes and responsibilities

# 1. Introduction

This overview describes version 5 of the Catan Game Project Gen3. Earlier versions focused mainly on constructing the playboard, rendering the scoreboard, enabling human Initial Placement, and adding stronger AI placement algorithms (AI = Artificial Intelligence, or simply bots). Version 5 keeps those foundations, but the main milestone is broader: the project now supports basic interaction during the Execution phase and provides a foundation for strategy-guided AI interaction during that phase.

The game is implemented in Python with Pygame. It models a Catan-style board with 19 land tiles, surrounding sea tiles, intersections, roads, harbors, resource production, and a visible scoreboard. The user interface is designed for a mixed human/AI game: the human player receives visual guidance and clickable choices, while AI players receive strategy-aware recommendations that can be executed step by step.

## Plain-English summary

The project no longer only asks: "Where should a player place the first settlements?" It now also asks: "Given the current hand, board, ports, risks, and strategy, what should this player do next?"

# 2. Project status in v5

- **Initial Placement:** Human and AI placement flows remain supported, including visual guidance and several AI placement algorithms.
- **Execution phase:** Human players can roll dice, resolve robber flow, trade with bank, buy development cards, and build roads, settlements, and cities through guided UI panels.
- **AI assistance:** AI players use a BEST NOW recommendation instead of blind action execution. Continue executes one selected action, then the system rescans.
- **Same-turn continuation:** After each successful action, the game refreshes legal actions and strategy context while keeping the same player in ActionSelection.
- **Dashboard upgrades:** The scoreboard now includes resource deltas, question-mark detail popups, trade-rate-aware resource counts, and v045-style development-card triplets.

The project is presented on GitHub as an educational AI/game-engine project rather than as a finished commercial Catan clone. Usage and distribution are governed by the repository’s MIT License. The current code demonstrates progressively richer gameplay mechanics, deterministic GUI flows, and strategy-oriented AI support.

# 3. System architecture

The project separates the board and game rules from the rendering and interaction layers. Version 5 adds a strategy/execution layer between the pure game state and the GUI.

| Layer           | Purpose                                                                                      | Representative modules                                         |
|-----------------|----------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Core model      | Represents the board, players, resources, roads, settlements, cities, ports, and game state. | core/board.py, core/player.py, core/game.py, core/constants.py |
| Action legality | Determines what actions are legal right now for the active player.                           | core/viable_action_scanner.py                                  |

| Layer                   | Purpose                                                                                       | Representative modules                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Strategy planning       | Scores candidate actions and links actions to current strategy direction.                     | core/action_planner.py, core/strategy_timing.py, core/resource_time_estimator.py |
| Execution orchestration | Runs selected actions, refreshes state, and supports AI Continue / human button flows.        | core/execution_phase_manager.py, core/game.py                                    |
| GUI and routing         | Draws the board, scoreboard, panels, guidance overlays, animations, and handles mouse clicks. | gui/gui.py, gui/event_handler.py, gui/gui_human_player.py                        |

This split keeps the project maintainable. Board state changes should happen in core logic. User interaction details should remain in GUI modules. Strategy modules should recommend actions, not directly draw on the screen.

## 4. GUI and interaction design

The GUI consists of four main areas: the playboard, the button panel, the scoreboard, and the right-side information panels. The playboard shows the hexagonal board, ports, roads, settlements, cities, robber, and animated guidance highlights. The button panel contains human actions such as Roll Dice, Trade with Bank, Buy Development Card, Build Road, Build Settlement, Build City, and End Turn.

| GUI module                   | Responsibility                                                                                                    |
|------------------------------|-------------------------------------------------------------------------------------------------------------------|
| gui.py                       | Central rendering engine for board, permanent pieces, scoreboard, turn-detail popup, and general drawing helpers. |
| gui_guidance.py              | Initial Placement guidance state machine.                                                                         |
| gui_human_player.py          | Button-panel rendering and active/inactive button state.                                                          |
| gui_human_road_guidance.py   | Execution-phase road build guidance for human players.                                                            |
| gui_human_buy_guidance.py    | Execution-phase city and settlement guidance for human players.                                                   |
| gui_trade_bank_panel.py      | Trade-with-bank panel, resource give/get selection, and viable trade execution.                                   |
| gui_execution_debug_panel.py | AI and strategy debug information, including BEST NOW.                                                            |
| event_handler.py             | Click routing between buttons, modal panels, board guidance, robber flow, and confirmation icons.                 |
| gui_constants.py             | Positioning constants, fonts, colours, images, and sound registry.                                                |

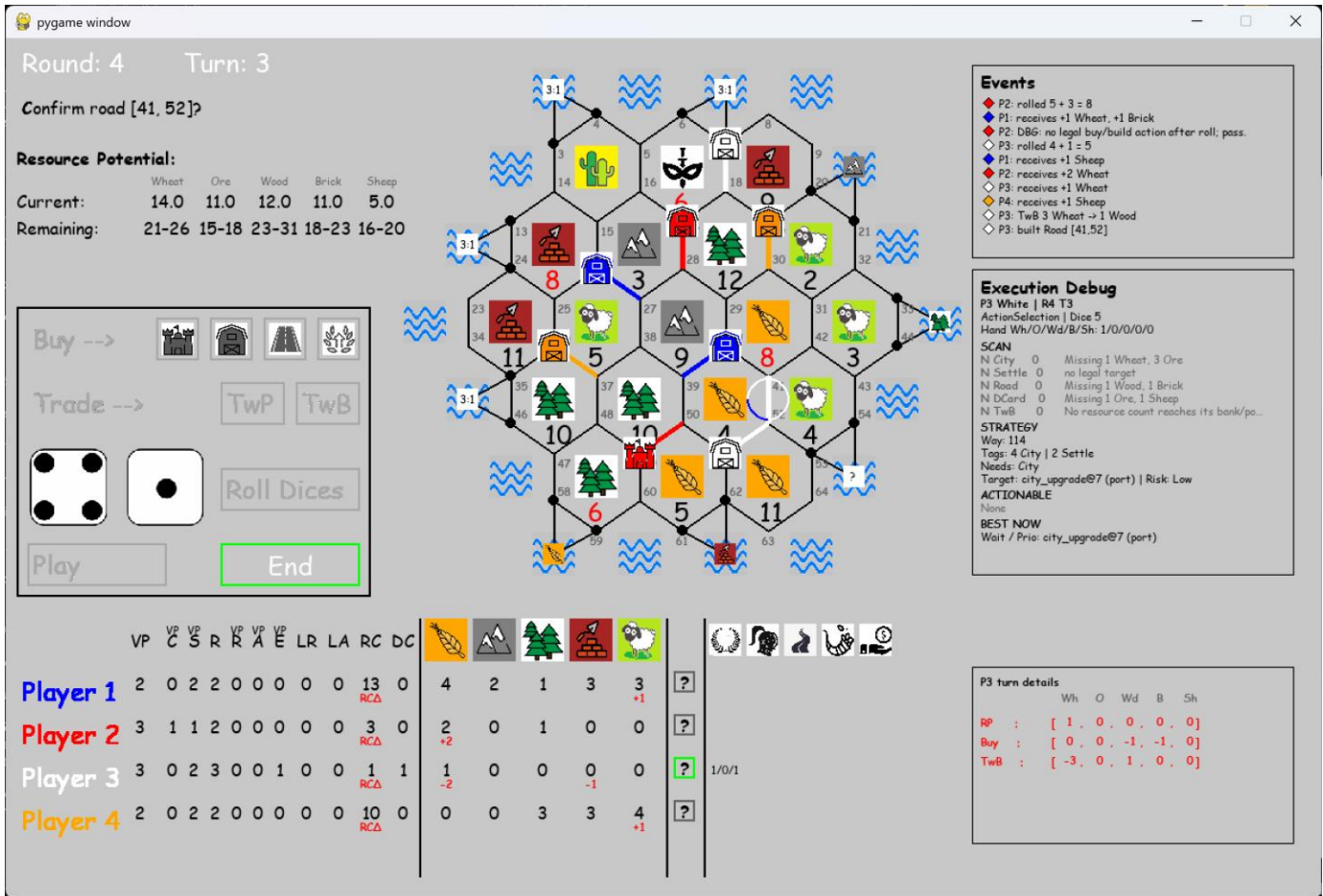


Figure 2. Existing Gen3 GUI rendering Playboard, Buttons, Scoreboard, Events, Turn Details (pop-up) and Execution Debug panel.

## 5. Human gameplay flow

Human gameplay in v5 is designed around guided, one-action-at-a-time interaction. The human player clicks an active button, sees only legal choices, confirms a target if needed, and then returns to ActionSelection with freshly updated buttons.

1. Roll dice.
2. Resolve resource production or robber flow if a 7 is rolled.
3. Enter ActionSelection for the same player.
4. Choose one action: Trade with Bank, Build Road, Build Settlement, Build City, Buy Development Card, or End Turn.
5. If the action needs a board target, show highlighted legal targets and wait for the player to select one.
6. Execute the action, update the board, update resources, play sound/animation, and run "Layer 4" (see next chapter).
7. Return to ActionSelection for another possible action, or End Turn.

This flow avoids stale buttons and stale recommendations. Each action is treated as a state-changing event. After that event, the game no longer trusts the old scan; it recomputes what is legal and useful.

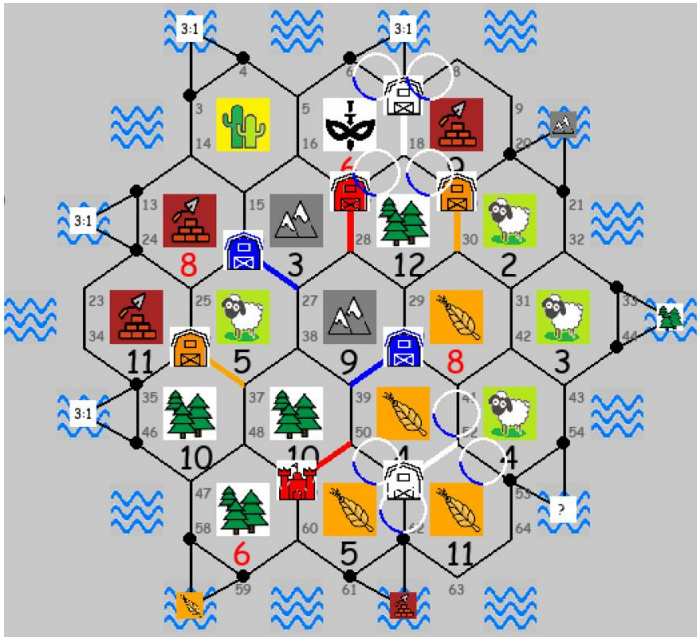


Figure 3. Road-building guidance for human player P3 (white), showing highlighted legal road choices.

## 6. Layered execution model

The execution architecture is easiest to understand as four layers. Each layer has one clear responsibility.

| Layer | Name                   | Responsibility                                                                                                                                    |
|-------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1     | Viable Action Scanner  | Determines what is legally possible now: trade, build, buy, robber, roll, or end turn.                                                            |
| 2     | Strategy Matching      | Matches legal candidates with the player strategy and computes the BEST NOW action.                                                               |
| 3     | Execution Routing      | Routes the selected action through AI Continue or human UI guidance.                                                                              |
| 4     | Same-Turn Continuation | After one successful action, refreshes strategy context, legal actions, BEST NOW, buttons, and debug panels while keeping the same player active. |

### Why Layer 4 matters

In Catan, order matters. A bank trade may unlock a city. A road may unlock a settlement. A settlement may change port access. Therefore v5 executes one action, refreshes, and then decides again.

## 7. Strategy and timing engine

Version 5 separates Initial Placement intelligence from Execution-phase intelligence. Initial Placement still supports pip, port, five-strategy, and Markov-style evaluation. Execution-phase planning uses Expected-Hand timing and strategy continuation scoring to estimate how many own turns are needed to reach useful goals. This version is still a foundation: the strategy and timing engine is useful, but there is still room for significant enhancement.



- **Expected-Hand timing:** Estimates resource arrival time from the player hand, production pips, ports, and trade assumptions.
- **Strategy requirements:** Uses a structured list of strategic resource requirements to compare paths such as road building, city upgrades, development-card buying, and resource coverage.
- **Continuation scoring:** After a possible action, estimates the best continuation strategy and total expected own turns.
- **Scarcity and viability:** Flags scarce resources, monopoly risk, and strategies that are unrealistic for a player because of missing access or weak production.

The practical result is that an action is not only legal; it can also be strategically useful. BEST NOW is meant to be the bridge between the rules engine and the strategy engine.

## 8. Trade with Bank and ports

Trade with Bank is now modeled as a vector-based action. Each player has a resource-specific trade-rate vector, for example [4, 4, 4, 4, 4] at the start of the game. Harbors update this vector: a 3:1 port improves all rates to 3 where relevant, while a 2:1 resource port improves the matching resource only.

- The TwB panel displays give/get choices and executes only viable trades.
- A successful TwB trade can immediately unlock a real action, such as building a city or road.
- For AI, TwB is treated as a support action: useful only when it directly enables the strategic follow-up action.

The screenshot displays the Catan Game Project Gen3 interface. At the top, it shows 'Round: 2' and 'Turn: 3'. The 'Resource Potential' section lists current and remaining resources for each player. The central part of the screen shows a hexagonal game board with various resources and actions. On the left, there are buttons for 'Buy -->', 'Trade -->', 'Roll Dices', 'Play', and 'End'. The bottom left shows a table of player statistics for four players. The bottom right features the 'Trade with Bank' panel, which allows players to give or get resources at specific rates. The right side of the screen contains an 'Events' log and an 'Execution Debug' window.

**Resource Potential:**

|            | Wheat | Ore   | Wood  | Brick | Sheep |
|------------|-------|-------|-------|-------|-------|
| Current:   | 14.0  | 11.0  | 12.0  | 11.0  | 5.0   |
| Remaining: | 21-26 | 15-18 | 23-31 | 18-23 | 16-20 |

**Trade with Bank Panel:**

| Resource    | Give | Get |
|-------------|------|-----|
| Wheat (3:1) | 3    | 0   |
| Ore (3:1)   | 0    | 0   |
| Wood (3:1)  | 0    | 0   |
| Brick (3:1) | 0    | 0   |
| Sheep (3:1) | 0    | 1   |

**Events Log:**

- P1: targets P2 for steal
- P1: steals Wood from P2
- P1: DB6: Road [15, 26] legal but skipped; preferred City; pass.
- P2: rolled 2 + 2 = 4
- P1: receives +1 Wheat
- P3: receives +1 Wheat
- P2: DB6: no legal buy/build action after roll; pass.
- P3: rolled 6 + 5 = 11
- P3: receives +1 Wheat
- P4: receives +1 Brick

**Execution Debug:**

P3 White | R2 T3  
ActionSelection | Dice 11  
Hand Wh/O/Wd/B/Sh: 5/1/0/0/0

SCAN

|          |   |                         |
|----------|---|-------------------------|
| N City   | 0 | Missing 2 Ore           |
| N Settle | 0 | no legal target         |
| N Road   | 0 | Missing 1 Wood, 1 Brick |
| N DCard  | 0 | Missing 1 Sheep         |
| Y TwB    | 4 | best 3 Wheat->Ore       |

STRATEGY

Way: 117  
Tags: LR | 4 City | 1 Settle  
Needs: City  
Target: city\_upgrade@7 (port) | Risk: Low  
ACTIONABLE  
None  
BEST NOW  
Wait / Prio: city\_upgrade@7 (port)

**Player Statistics Table:**

|          | VP | C | S | R | R | A | E | LR | LA | RC | DC |
|----------|----|---|---|---|---|---|---|----|----|----|----|
| Player 1 | 2  | 0 | 2 | 2 | 0 | 0 | 0 | 0  | 0  | 7  | 0  |
| Player 2 | 2  | 0 | 2 | 2 | 0 | 0 | 0 | 0  | 0  | 6  | 0  |
| Player 3 | 2  | 0 | 2 | 2 | 0 | 0 | 0 | 0  | 0  | 6  | 0  |
| Player 4 | 2  | 0 | 2 | 2 | 0 | 0 | 0 | 0  | 0  | 6  | 0  |

Figure 4. Trade with Bank panel, showing give/get resource choices and viable bank trade execution.

## 9. Development-card dashboard

The scoreboard now uses a v045-style development-card detail display. Instead of showing one compact total under each card type, the dcard area can show a triplet:

new / playable / played-or-VP

Example:

Knight: 1/2/0

This means one knight was bought this turn, two knights are playable in a later turn, and zero knights have been played. For victory-point cards, the third value represents the VP count rather than a normal played-card count.

- Human-player rows show detailed dcard triplets.
- Opponent dcard identities remain hidden during normal play.
- When development/debug analysis is enabled, all player dcard triplets can be shown.
- The question-mark turn-detail popup is aligned with the right-side panels so it no longer covers the dcard statistics area.

|                 | VP | C | S | R | A | E | LR | LA | RC | DC |   |   |   |   |   |   |   |       |  |
|-----------------|----|---|---|---|---|---|----|----|----|----|---|---|---|---|---|---|---|-------|--|
| <b>Player 1</b> | 2  | 0 | 2 | 2 | 0 | 0 | 0  | 0  | 0  | 13 | 0 | 4 | 2 | 1 | 3 | 3 | ? |       |  |
| <b>Player 2</b> | 3  | 1 | 1 | 2 | 0 | 0 | 0  | 0  | 0  | 3  | 0 | 2 | 0 | 1 | 0 | 0 | ? |       |  |
| <b>Player 3</b> | 3  | 0 | 2 | 2 | 0 | 0 | 1  | 0  | 0  | 3  | 1 | 1 | 0 | 1 | 1 | 0 | ? | 1/0/1 |  |
| <b>Player 4</b> | 2  | 0 | 2 | 2 | 0 | 0 | 0  | 0  | 0  | 10 | 0 | 0 | 0 | 3 | 3 | 4 | ? |       |  |

Figure 5. Development-card triplet details in the scoreboard.

## 10. AI execution flow

AI execution is deliberately stepwise. The Continue button executes one BEST NOW action, then “Layer 4” refreshes the state. The AI does not blindly perform every viable action in one click.

- AI rolls or enters ActionSelection.
- The scanner finds legal actions.
- The strategy layer selects BEST NOW.
- Continue executes one action.
- “Layer 4” rescans and recomputes.
- The next Continue executes the next BEST NOW action or ends the turn if no useful action remains.

This makes AI behavior explainable and debuggable. A user can see why the AI chose a city, road, settlement, development card, bank trade, or pass action.



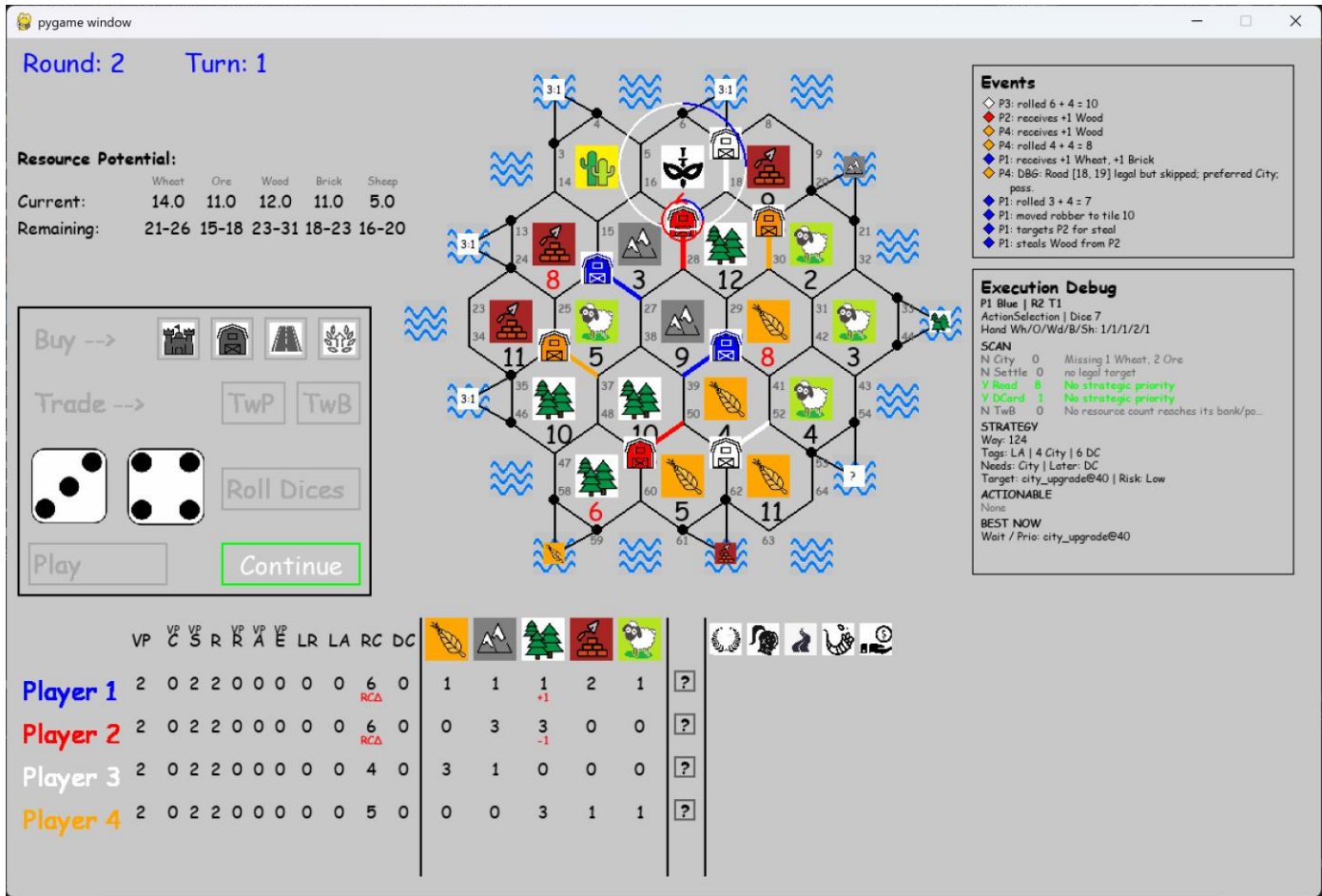


Figure 6. AI Player 1 (blue) rolled a 7, triggering robber guidance and the Continue flow.

## 11. Sound and animation polish

Sound and animation are used as feedback, not as game logic. Successful city, settlement and road builds are supported by sound; same for dice rolls being “7”, stealing, successful bank trades and development-card purchases.

The guidance system uses pulsing highlights for legal targets, confirmation icons for selected targets, and animation cleanup rules so old robber/steal visuals do not overlap with city, settlement, or road guidance.

## 12. Known limitations and next steps

- **README refresh:** After Overview\_v5 is finalized, the README should be updated to link to docs/Overview\_v5.pdf instead of Overview\_v4.pdf.
- **TwP (Trade with Player):** Trade with Bank is implemented and polished; player-to-player trade is currently not implemented and will be added later in time.
- **Development card play:** Buying and dashboard tracking are supported. Playing development cards is currently not implemented and will be added later in time.
- **AI learning:** Future versions may connect rule-based strategy planning using ML algorithms.

## 13. Acknowledgements and external inspiration

This project was developed as an educational AI/game-engine project and builds on several sources of inspiration.

First, I would like to acknowledge Lauren Nagel's work, Analysis of "The Settlers of Catan" Using Markov Chains. That paper helped inspire the project's early exploration of Markov-style evaluation for Initial Placement. In this project, that idea was adapted into a practical placement-evaluation approach suitable for a Python/Pygame implementation.

Second, I would like to acknowledge the strategy articles published by Player One on BoardGameAnalysis.com, especially the series The 102/143 Ways to Win at Catan, Parts I, II and III. These articles helped shape the broader way of thinking about Catan strategy: not only as isolated moves, but as different victory paths with different resource needs, risks, and timing profiles.

The implementation in this repository is my own educational adaptation. The referenced works served as inspiration for strategic thinking, probability-based evaluation, and the organization of possible paths to victory; they are not copied code or direct implementations.

## Annex A. Module overview

| Module                           | Short description                                                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| core/board.py                    | Defines tiles, intersections, roads, harbors, legal board geometry, and connectivity.                                                   |
| core/player.py                   | Stores resources, pieces, roads, cities, settlements, development cards, trade rates, and player statistics.                            |
| core/game.py                     | Central orchestrator for phases, turn state, actions, resource production, robber flow, strategy dashboard, and "Layer 4" continuation. |
| core/constants.py                | Stores game constants, resource costs, image/sound identifiers, configuration flags, and helper definitions.                            |
| core/viable_action_scanner.py    | Scans legal actions available to the current player in the current state.                                                               |
| core/action_planner.py           | Scores possible actions and estimates continuation value after an action.                                                               |
| core/strategy_timing.py          | Builds timing reports for strategy requirements and resource needs.                                                                     |
| core/resource_time_estimator.py  | Expected-Hand engine for estimating resource acquisition timing.                                                                        |
| core/execution_phase_manager.py  | Coordinates execution phase actions and AI Continue behavior.                                                                           |
| gui/gui.py                       | Main renderer for board, scoreboard, turn details, buttons integration, and drawing helpers.                                            |
| gui/gui_human_player.py          | Renders and updates human action buttons.                                                                                               |
| gui/event_handler.py             | Routes clicks to buttons, board guidance, panels, robber flow, and confirmations.                                                       |
| gui/gui_human_road_guidance.py   | Human road-building target guidance during Execution.                                                                                   |
| gui/gui_human_buy_guidance.py    | Human city/settlement guidance during Execution.                                                                                        |
| gui/gui_trade_bank_panel.py      | Human Trade with Bank panel and interactions.                                                                                           |
| gui/gui_execution_debug_panel.py | Strategy/BEST NOW debug display.                                                                                                        |

## Annex B. Key classes and responsibilities

| Class             | Responsibility                                                                                                                  |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Game              | Owns the live game state, current phase, turn, players, board, action execution, resource production, and continuation refresh. |
| Board             | Defines the board graph: tiles, intersections, roads, ports, legal paths, and adjacency.                                        |
| Player            | Tracks a player hand, pieces, roads, cities, settlements, ports, trade rates, development cards, and victory points.            |
| Tile              | Represents one hex tile with resource type, dice number, robber state, and corners.                                             |
| Intersection      | Represents one buildable vertex with occupancy, port status, and neighboring roads.                                             |
| Road              | Represents one edge between two intersections and its owner/type.                                                               |
| TurnDetails       | Tracks resource deltas and one-turn event details for scoreboard and question-mark popup.                                       |
| StrategyDashboard | Summarizes strategy-relevant player statistics such as resource cards, dcard count, VP, route, and army values.                 |
| GUI               | Draws the game view, scoreboard, animations, and detail panels.                                                                 |
| GUIHumanPlayer    | Draws human-player action buttons and manages their active/inactive state.                                                      |
| EventHandler      | Routes user clicks to the correct current interaction.                                                                          |

## Publication note

This document is prepared as the technical overview for GitHub publication. The exported PDF can be linked from the README as docs/Overview\_v5.pdf.